

Introduction to the course

Welcome to a course in the module Analysis and insights in Hypergene. In this course you'll learn how to integrate data into Hypergene and use it to build a report. We will be working with data that has already been cleaned and we'll therefore only do the Load part of the ETL process. We will, however, talk about Extracting and transforming data too.

Prerequisites: You should have basic knowledge of SQL and some knowledge about the Hypergene node tree (there is a separate course about that).

The course will cover the following areas:

1. Integrating data into Hypergene
2. Create a report
3. Create basic report objects in the report

The course is very practical and you should have access to some sort of Hypergene environment. We will be working with a fictional use case that will be presented later in the course.

Overview data integration

Usually data is integrated into Hypergene in three steps. All steps except for the last one are very general data extraction and transformation that could be the same for any BI-platform.

First we extract data from the source system into the Hypergene database (or a stage database). The next step is the adapter where data is transformed to fit the system neutral stage environment. From these stage tables (or views) the data is integrated into Hypergene.

A system neutral stage environment is the tables structured to work for all for example financial data (no matter which source system the financial data comes from).

General guidelines the node-tree

Before taking this course you should have some basic knowledge on what you can do (**and should not do**) in the node tree. There is a supplement document to this called nodetree.pdf

Introduction to customer case and stage data

Now the customer case will be presented and the stage data that we have to work with.

The goal of this part of the course is to

1. Get familiar with the customer case
2. Examine the data in the stage database

Requirements

Show sales quantity:

Show sales quantity for each product, sales rep and country

Show sales amount:

Show sales amount for each product, sales rep and country

Filter the sales data based on year and months :

The user should be able to make selection in the report that filters the data on year and month.

Filter the sales data based on organisation:

The user should be able to make selection in the report that filters the data on organisation.

Show the underlying transactions:

Show the underlying transaction for the sales data

Compare last year's sales amount to this year's:

Compare the sales amount this month to the same month previous year.

The report should always show current year and month as default:

When the report is opened the current month and year should be selected

Filter the transactions on dates:

The user should be able to filter the transactions

Limit which parts of the organisation the user can see:

Users should only see the part of the organisation that they are responsible for.

Show accumulated sales amount:

Show sales amount and sales quantity for each product, sales rep and country

Customer data

This should be already loaded in the database of lab environment. \

Hypergene integration

The Hypergene integration usually consists of three steps:

1. Node tree: Set up the structure that is used to decide what data should be gotten from the database.
2. Database: Create a procedure ([neutral_schema].[sp_run_integration])) that runs the procedures sp_eos_integration with parameters that point to the structure in the node tree (and other parameters that determines how the integration should be done).
3. SQL Server Agent: This is where the procedure is scheduled so that it runs every night. We won't schedule any integrations since this is only a lab environment.

The goal with the following activities is to:

1. Create the structures needed in the node tree

2. Integrate data to Hypergene

Since we need the dimensions before we run the fact integration we'll start by setting up the dimensions we need. The next step is to run the fact integration.

In the node tree the integration structure is set up. Hypergene integration consists of two parts:

- Dimension integration
- Fact integration

Under the respective node dimensions and/or facts are configured. We tend to always name the dimensions and fact tables with a prefix that tells us which data-model they belong to.

Database:

The first step to scheduling the integration is to create a procedure ([neutral_schema].[sp_run_integration])) that runs the procedures sp_eos_integration with parameters that point to the structure in the node tree (and other parameters that determines how the integration should be done).

Data should when possible be integrate incrementally (only get the new data), this is however not always possible since it requires that there has to be a way of know if a row is new in the source data.

The alternative to incremental integration is a full integration which means that all data is truncated and the data from the source system is added again.

That is usually one SQL agent job for each data model i Hypergene and each job should consist of the following steps:

1. Load data to the adapter ([schema].[sp_import_data]).
2. Freeze history (if needed) ([schema].[sp_history_save]).
3. Load data to the stage table ([schema].[sp_load_stage]).
4. Integrate data into Hypergene (both dimensions and facts) ([neutral_schema].[sp_run_integration]).
5. If another module is dependent on the data from the model, trigger the next SQL Server Agent job.

System administration in the node tree

In this section we'll shortly go through how to log into Hypergene and open the system administration.

Log in

Log into Hypergene by writing username and password (rb/rb). When you have logged in you'll be greeted by the portal page.

Open the system administrationen

Once you've logged in to Hypergene, you can open System Administration from the administration menu. Hover your mouse pointer over  (cogwheel) in the left corner and select System Administration. System Administration will then open in a new tab in your browser.

Open the node tree

In the start menu you can reach many different things, but here we'll focus on the node tree. Open it by clicking on **Start -> Node tree**.

Hypergene integration

Hypergene integration can be found in two places in the labenvironment (and in many other Hypergene environments). In all Hypergene installation it can be found under sys, but sometimes it has also been added directly under root (for easier access).

Important Nodes:

Dimensions:

This is where you'll find all dimensions in Hypergene. Dimensions can either be created manually, directly under the dimensions node, or through so called dimensions Integration (which we'll talk about later in this course).

Hypergene-Integration

This is where you'll find the structures that define how data should be integrated into Hypergene.

Tables

This is where you'll find all tables (facts) in Hypergene.

Facts can, like dimensions, both be integrated and also created manually. Tables are usually only created manually if they are to be used to enter data in Hypergene or if they are based on a database view (using Integrated Hypergene).

Dimensions integration

The dimensions integration in Hypergene will get data from the defined datasource and add it to Hypergene table r2, r2_specific, links and nodes. If the dimensional members have properties these are added to the property table textValues, intValue, floatValue och dateValues. The following chapter will go through these.

The Hypergene database

There are 7 important tables concerning the Hypergene node tree, that might be good to know:

1. **nodes:** stores information about all of the nodes (like title and brief)

2. **links**: stores information about relationships between nodes (one level, parent and child)
3. **r2**: stores information about relationships between nodes (all levels, parent, child, grandparents, great grandparents etc.)
4. **textValues**: stores properties that a node has if they are text
5. **floatValues**: stores properties that a node has if they are numbers with decimals.
6. **intValues**: stores properties that a node has if they are numbers without decimals
7. **dateValues**: stores properties that a node has if they are dates.

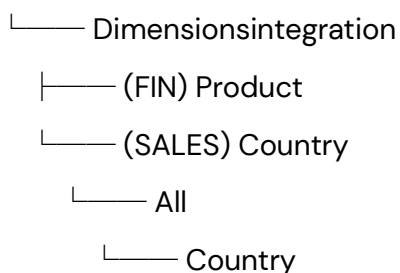
There are also r2_specific tables (one for each dimension) and they are always called:

1. **r2_dimension (example: r2_account)**: stores information about relationships between nodes (all levels, parent, child, grandparents, great grandparents etc.) for one specific dimension.

How to do Dimension Integration ?

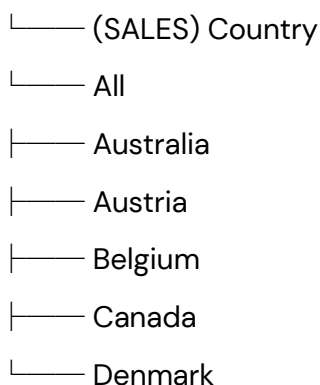
In the dimensions integration group in the node tree, you set up the structure used to integrate dimensions into Hypergene.

Hypergene-Integration



The resulting structure looks like this.

root



Configuration Guide – Creating the (SALES) Country Dimension

Step 1: Create (SALES) Country

Navigate to

root

└── Hypergene-integration

└── Dimensionsintegration

└── (SALES) Country

Set

Title: (SALES) Country

Key: sales_dim_country

Action

Save changes.

Purpose

Create the dimension node and assign a unique key.

Step 2: Configure Data Collection for (SALES) Country

Navigate to

root

└── Hypergene-integration

└── Dimensionsintegration

└── (SALES) Country

Tab

Dimensionintegration

Set

dim: sales_dim_country

dbname: [sales_system].[dbo].[stage_dim_country]

Action

Save changes.

Purpose

Define the dimension name and the source database table used for data retrieval.

Step 3: Create the All Level

Navigate to

root

└── Hypergene-integration

└── Dimensionsintegration

└── (SALES) Country

└── All

Set

Title: All

Key: all

Action

Save changes.

Purpose

Create the first level under the dimension.

Step 4: Configure the All Group

Navigate to

root

└── Hypergene-integration

└── Dimensionsintegration

└── (SALES) Country

└── All

Tab

Dimensionintegration

Set

al_name: All

Action

Save changes.

Purpose

Specify the display name for the all-group.

Step 5: Create the Country Node

Navigate to

root

└── Hypergene-integration

└── Dimensionsintegration

└── (SALES) Country

└── All

└── Country

Tab

Country

Set

Title: Country

Action

Save changes.

Purpose

Create the Country member node beneath the All level.

Step 6: Configure Data Collection for Country

Navigate to

root

└── Hypergene-integration

└── Dimensionsintegration

└── (SALES) Country

└── All

└── Country

Tab

system

Set

source_key: country

title_dbname: country

brief_dbname: country

alias_dbname: country

Action

Save changes.

Purpose

Use the country column as the source key, title, brief description, and alias.

Step 7: Run the Integration

Navigate to

root

└── Hypergene-integration

└── Dimensionsintegration

└── (SALES) Country

Execute

dbo.sp_eos_Integration

Parameters

@dimensions_insert = 1

@create_dimensions = 1 (only when creating the dimension for the first time)

@set_id = '\$38886'

Purpose

Create and populate the dimension members from the configured source table.

Note: Use @create_dimensions = 1 only during the initial creation of the dimension. For subsequent runs, the parameter is not required.

Fact integration

During the fact integration data is copied from stage-tables to the Hypergene database. As they happens dimensional values are translated to the nod id that member has in Hypergene. For example Account 1411 will be translated to the nod id that account 1411 has in Hypergene. This is why you have to integrate all dimension before integrating the facts in Hypergene.

Fact integration: Sales data

Note! In the lab environment, you can already find (SALES) Sales data SOLUTION. This is, as the name suggests, an already set-up dimension for Sales data. You should, however, set up your own integration for practice, but you can always check the solution if you need help.

1. Create a node under faktaintegration (fact integration) and give it the title (SALES) Sales data
2. In the tab fact integration add which table data should be collected from (source_dbname) and what the fact-table in Hypergene should be called.
3. Read more about the different parameters by pressing **+**

When the parent-node ((SALES) Sales data) has been created you can add one child for each column you want in the table.

Fact Integration Guide – (SALES) Sales Data

Step 1: Configure ORDER NUMBER

Navigate to

root

└── Hypergene-integration

└── Faktaintegration

└── (SALES) Sales data

└── ORDER NUMBER

Tab

Faktaintegration

Set

source_dbname: ORDERNUMBER

column_type: 3

column_name: ORDERNUMBER

Step 2: Configure QUANTITY ORDERED

Navigate to

root

└── Hypergene-integration

- └── Faktaintegration
- └── (SALES) Sales data
- └── QUANTITY ORDERED

Tab

Faktaintegration

Set

source_dbname: QUANTITYORDERED

source_datatype: NUMERIC(28,8)

column_type: 2

column_name: QUANTITYORDERED

Step 3: Configure SALES

Navigate to

root

- └── Hypergene-integration
- └── Faktaintegration
- └── (SALES) Sales data
- └── SALES

^^

Tab

Faktaintegration

Set

source_dbname: SALES

source_datatype: NUMERIC(28,8)

column_type: 2

column_name: SALES

Step 4: Configure PERIOD

Navigate to

root

- └── Hypergene-integration

└── Faktaintegration
└── (SALES) Sales data
└── PERIOD

Tab

Faktaintegration

Set

source_dbname: MONTH_ID

dimension: period

Step 5: Configure YEAR PERIOD

Navigate to

root

└── Hypergene-integration
└── Faktaintegration
└── (SALES) Sales data
└── YEAR PERIOD

Tab

Faktaintegration

Set

source_dbname: year_period

dimension: year_period

Step 6: Configure YEAR

Navigate to

root

└── Hypergene-integration
└── Faktaintegration
└── (SALES) Sales data
└── YEAR

Tab

Faktaintegration

Set

source_dbname: YEAR_ID

dimension: year

Step 7: Configure PRODUCT CODE**Navigate to**

root

└── Hypergene-integration

└── Faktaintegration

└── (SALES) Sales data

└── PRODUCT CODE

Tab

Faktaintegration

Set

source_dbname: PRODUCTCODE

column_type: 3

column_name: PRODUCTCODE

Step 8: Configure COUNTRY**Navigate to**

root

└── Hypergene-integration

└── Faktaintegration

└── (SALES) Sales data

└── COUNTRY

Tab

Faktaintegration

Set

source_dbname: COUNTRY

dimension: sales_dim_country

Step 9: Configure SALES PERSON

Navigate to

root

└── Hypergene-integration

└── Faktaintegration

└── (SALES) Sales data

└── SALES PERSON

Tab

Faktaintegration

Set

source_dbname: sales_person

dimension: hg_resource

column_type: 1

column_name: SALES_PERSON

Step 10: Configure DEAL SIZE

Navigate to

root

└── Hypergene-integration

└── Faktaintegration

└── (SALES) Sales data

└── DEAL SIZE

Tab

Faktaintegration

Set

source_dbname: DEALSIZE

column_type: 3

column_name: DEALSIZE

Step 11: Configure ORGANISATION

Navigate to

root

└── Hypergene-integration

└── Faktaintegration

└── (SALES) Sales data

└── ORGANISATION

Tab

Faktaintegration

Set

source_dbname: organisation

dimension: plan_org

column_type: 1

column_name: ORGANISATION

Step 12: Configure PRODUCT

Navigate to

root

└── Hypergene-integration

└── Faktaintegration

└── (SALES) Sales data

└── PRODUCT

Tab

Faktaintegration

Set

source_dbname: product

dimension: fin_product

column_type: 1

column_name: PRODUCT

Step 13: Run the Fact Integration

Navigate to

root

└── Hypergene-integration

└── Faktaintegration

└── (SALES) Sales data

Execute

dbo.sp_eos_Integration

Parameters

@dimensions_insert = 0

@dimensions_update = 0

@dimensions_delete = 0

@dimensions_reorder = 0

@create_dimensions = 0

@fact_insert = 1

@create_fact_tables = 1 (only the first time)

@fact_incremental = 0

@set_id = '\$38551'

@debug = 0

@trace = 0

@nohistory = 0

@language_code = 'sv_SE'

Save changes after each configuration step before running the integration.

Step 14: Verify Table Registration

Navigate to

Start

└── Tables

Verify

Search for:

(SALES) Sales data

Open the table by clicking the **Edit (pencil)** icon.

If the table is not found, reload the node tree and search again.

Step 15: Verify Data Load

Run the following query:

```
SELECT *  
FROM dbo.fact_sales;
```

Expected Result

The query should return rows from `dbo.fact_sales`, confirming that the integration completed successfully and data was loaded into the fact table.